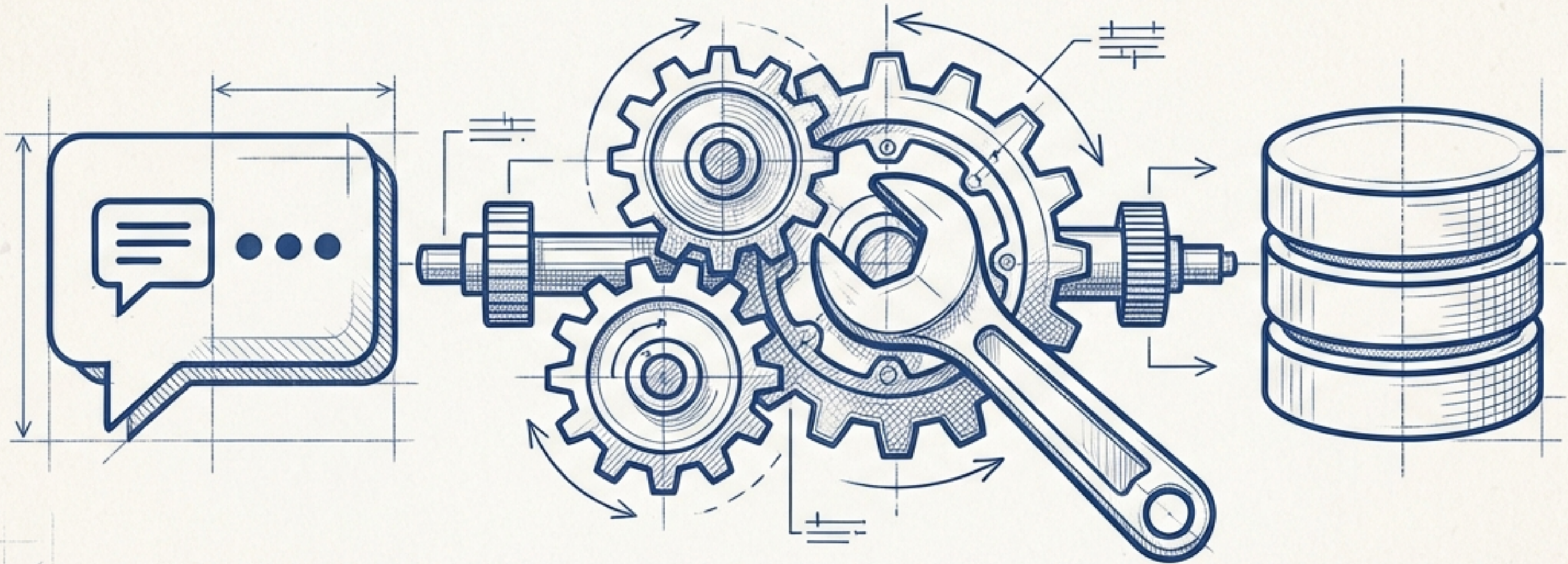


Práctica 1: Tu Primer Asistente con Herramientas

Implementando Function Calling en "FlightAI"



Bienvenido al laboratorio. Hoy transformaremos un chatbot estándar en un asistente útil capaz de interactuar con datos reales.

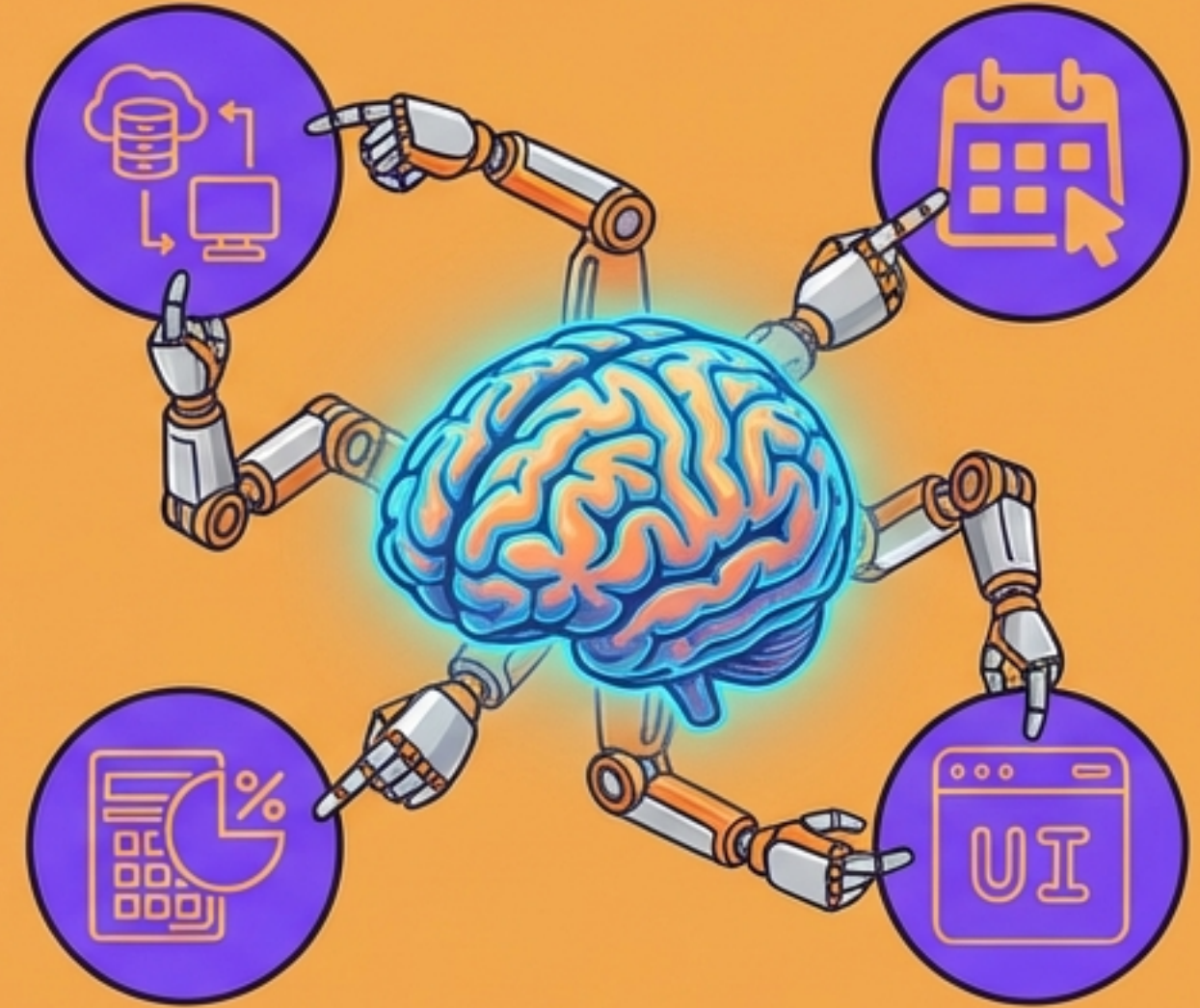
El Problema: Cerebros Aislados vs. Manos Útiles

El Cerebro Aislado



Los LLMs como GPT-4 son excelentes conversando, pero viven aislados. No pueden acceder a datos privados ni saber qué hora es por sí mismos.

La Solución: Function Calling

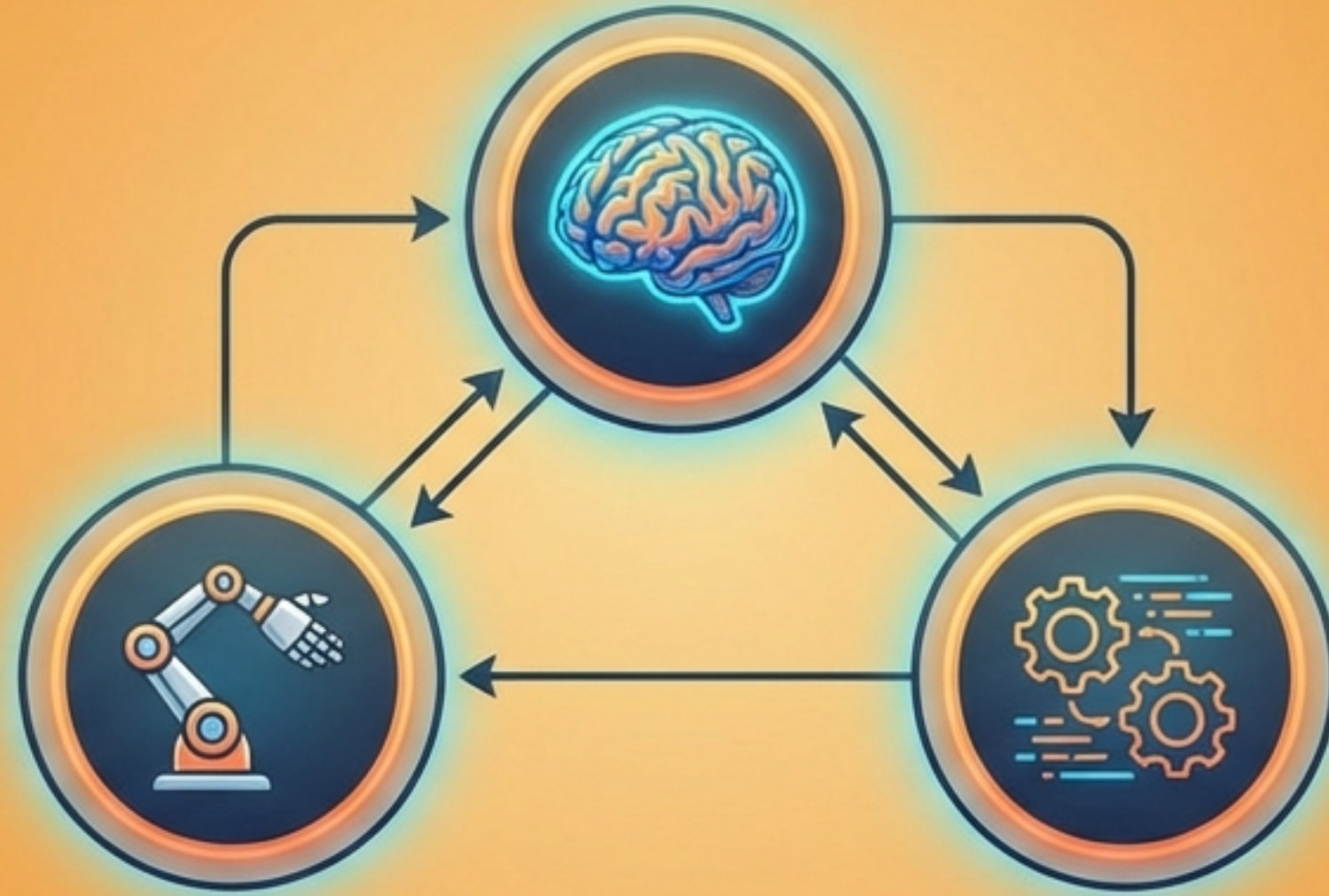


Function Calling dota a la IA de 'manos' para usar herramientas externas. La IA no ejecuta el código; decide cuándo pedirte a ti que lo ejecutes.

El Plano Arquitectónico de FlightAI

La Herramienta (Python)
Lógica de búsqueda

El Capataz (Handler)
Ejecución de órdenes



El Plano (Schema JSON)
Instrucciones para IA

Objetivo: Construir un Asistente de Aerolínea que consulta una base de datos de precios.
Flujo: Definir → Describir → Ejecutar.

Paso 1: Los Materiales (Datos y Lógica)

1. Base de datos ficticia

```
ticket_prices = {  
    "london": "$799",  
    "paris": "$899",  
    "tokyo": "$1400",  
    "berlin": "$499"  
}
```

2. Función Python real

```
def get_ticket_price(destination_city):  
    print(f"Tool called for {destination_city}")  
    city = destination_city.lower()  
    return ticket_prices.get(city, "Unknown")
```

**Manejo automático de
mayúsculas/minúsculas.**

Copia este código en tu sección de herramientas.
Esta función simula nuestra consulta a la base de datos.

Paso 2: El Manual de Instrucciones (Schema)

El LLM no puede leer tu código Python. Necesita una descripción en JSON para entender qué hace la herramienta y qué parámetros necesita.

```
def get_ticket_price(destination_city):
```

Traducción de
Parámetro

```
price_function = {  
  "name": "get_ticket_price",  
  "description": "Get the price of a return ticket...",  
  "parameters": {  
    "type": "object",  
    "properties": {  
      "destination_city": {  
        "type": "string",  
        "description": "The city that the customer wants to travel to"  
      }  
    },  
    "required": ["destination_city"]  
  }  
}
```

Copia este código en tu sección de herramientas.
Esta función simula nuestra consulta a la base de datos.

Paso 2 (Cont.): Registro de Herramientas

```
tools = [{  
    "type": "function",  
    "function": price_function}  
}]
```

Esta lista **'tools'** es la que pasaremos a la API de OpenAI más adelante. Sin este registro, el 'manual de instrucciones' se queda en un cajón y la IA no sabrá que la herramienta existe.

Paso 3: El Puente (Decodificar la Solicitud)

Cuando la IA quiere usar una herramienta, nos manda un mensaje especial (String). Debemos convertirlo a un Diccionario.

```
# Dentro de handle_tool_call(message):
```

```
tool_call = message.tool_calls[0]
```

```
# DECODIFICACIÓN
```

```
arguments = json.loads(tool_call.function.arguments)
```

```
city = arguments.get('destination_city')
```

⚠ **CRÍTICO:** Transforma texto
'{"city": "Berlin"}' en objeto Python.

Paso 3: El Puente (Ejecución y Respuesta)

Ejecutamos la función y empaquetamos el recibo para la IA.

```
# Ejecutar  
price = get_ticket_price(city)
```

```
# Empaquetar
```

```
response = {  
    "role": "tool",  
    "content": json.dumps({"destination_city": city, "price": price}),  
    "tool_call_id": tool_call.id  
}
```

Indica que es un resultado de función

ID de rastreo obligatorio

Paso 4: El Ciclo de Vida de la Petición



Protocolo de Inspección: 3 Pruebas Obligatorias

Para dar la práctica por válida, tu código debe superar tres escenarios distintos. No envíes tu archivo .py hasta ver estos tres resultados.



La Ruta Feliz (Éxito)

```
{  
  "role": "tool", {"destination_city": city, "price": $499},  
}
```



El Caso Borde (Dato no disponible)

```
{  
  "content": "json_error",  
  "error": "Error"  
}
```



Identidad (System Prompt)

```
{  
  "response": "Validate la length of tu system prompt."  
}
```


Prueba 1: Éxito (Dato Existente)

Input Usuario: "How much is a ticket to Berlin?"

```
> Tool get_ticket_price  
called for Berlin
```

PASS

A return ticket to Berlin
costs \$499.

Confirma que el JSON se leyó bien y el precio (\$499) se recuperó del diccionario.

Prueba 2: Manejo de Errores (Dato No Disponible)

Input Usuario: "How much is a ticket to Madrid?"

```
> Tool get_ticket_price  
called for Madrid
```

I'm sorry, I don't have
price info for Madrid.

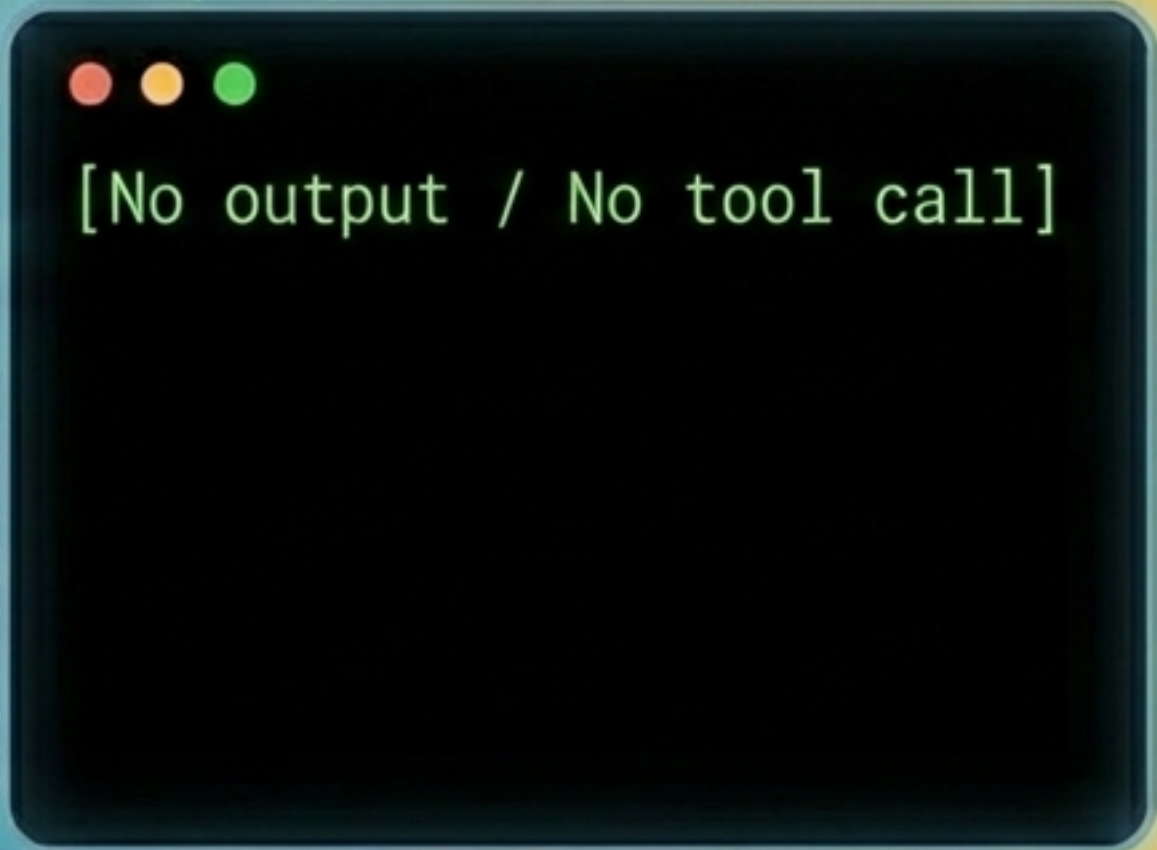


CRÍTICO: La función interna devuelve 'Unknown'. La IA **NO** debe inventar un precio.

CRÍTICO: La función interna devuelve 'Unknown'.
La IA **NO** debe inventar un precio.

Prueba 3: Personalidad e Identidad

Input Usuario: "Who are you?"

A dark-themed terminal window with three colored window control buttons (red, yellow, green) at the top left. It contains the text "[No output / No tool call]" in a green monospace font.

[No output / No tool call]

A light-colored chat window with three colored window control buttons (red, yellow, green) at the top left. It features a blue speech bubble containing the text "I am a helpful assistant for an Airline called FlightAI." Below the bubble is a white input field with a smiley face icon and an orange send button with a right-pointing arrow.

I am a helpful assistant for
an Airline called FlightAI.

Verifica que el system_message está activo y la IA es eficiente (no llama herramientas si no hace falta).

Guía de Solución de Problemas (Debugging)



Errores JSON

Revisa comas y llaves en `price_function`. Un error de sintaxis aquí romperá la llamada.



Alucinaciones

Si la IA inventa precios, revisa la “description” en el esquema. Debe ser clara sobre qué hace la herramienta.



Tipos de Datos

Asegúrate de que `handle_tool_call` siempre devuelve un string en el campo “content”.



Tip Pro: Usa `print()` en la terminal para ver los argumentos reales que envía la IA.

Misión Cumplida



Has logrado unir la Inteligencia Artificial Generativa (GPT) con Código Determinista (Python).

Acción: Una vez pasadas las 3 pruebas, sube tu archivo .py al campus virtual.